

Splitting Data into Training and Testing Sets

1. Introduction

In Machine Learning, we use data to train models that can make predictions. However, to evaluate how well our model performs on new, unseen data, we cannot test it on the same data used for training.

To solve this, we divide the dataset into two parts:

- Training set – used to learn the model.
- Testing set – used to evaluate how well the model generalizes.

تقسيم

تعلم الآلة حاضرة ٣
ذكاء شائع

2. Why Split the Data?

Imagine you memorize answers for a test — you might score perfectly on those questions but fail when faced with new ones.

The same happens with Machine Learning:

- If we test the model on the same data it learned from, it may overfit — meaning it memorizes the training data instead of learning patterns.
- Splitting helps us measure the model's ability to generalize to new situations.

حفظ من حاضرتهم !!

3. Typical Split Ratios

There is no fixed rule, but common ratios are:

Split Type	Training (%)	Testing (%)	Notes
Basic Split	70%	30%	Most common for small datasets
Moderate	80%	20%	When you have more data
Large Dataset	90%	10%	For very large datasets
With Validation	60%	20%	20% for testing + 20% validation

4. Example in Python (using scikit-learn)

```
from sklearn.model_selection import train_test_split
import numpy as np

# Example dataset
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8]]) # Features
y = np.array([2, 4, 6, 8, 10, 12, 14, 16]) # Labels

# Split data: 70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print("Training data:", X_train)
print("Testing data:", X_test)
```

Explanation:

- train_test_split divides data into training and testing sets.
- test_size=0.3 means 30% for testing.
- random_state=42 keeps the split the same every time you run it.

values still same

A. What is train_test_split function?

Train_test_split is a function from the scikit-learn library (sklearn.model_selection) used to divide your dataset into two or more parts:

- Training set → for learning (the model fits this data).
- Testing set → for evaluation (to check performance on unseen data).

B. Importing and Basic Syntax

```
from sklearn.model_selection import train_test_split
```

Syntax:

`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, shuffle=True)`

Parameter	Description
X	The input features. Example: hours studied, temperature, etc.
y	The target labels. Example: exam score, price, etc.
test_size	The percentage of data for testing. Example: 0.3 = 30% for testing and 70% for training.
train_size	Optional. You can also specify training size directly. قابل للإعادة
random_state	A fixed number to make the split reproducible (same random split every time).
shuffle	Whether to mix (shuffle) the data before splitting. Usually set to True.

سواء

Iris Dataset (مجموعة بيانات زهرة السوسن)**1. What is the Iris Dataset?**

The **Iris dataset** is a small but famous real dataset used for classification tasks. It was collected by the British biologist **Ronald A. Fisher** in 1936.

Each record describes a flower from one of **three species** of the iris plant:

1. *Iris setosa*
2. *Iris versicolor*
3. *Iris virginica*

نوع

The goal is to **predict the species** of an iris flower based on four measurements of its petals and sepals. بتلات

ملاحظات:

- كلمة **Iris** تعني زهرة السوسن، وهي نبات جميل متعدد الألوان.
- **Setosa** تعني "نحو شعيرات دقيقة".
- **Versicolor** تعني "متعدد الألوان" (لأن بتلاته تحتوي أكثر من لون).
- **Virginica** نسبة إلى ولاية فرجينيا (Virginia) في أمريكا، حيث وُجد هذا النوع.

2. Features (Inputs) : Each sample (row) contains four numeric features:

Feature	Description	Unit
Sepal length	The length of the outer part of the flower	cm
Sepal width	The width of the outer part	cm
Petal length	The length of the inner petal	cm
Petal width	The width of the inner petal	Cm

ملاحظات:

- **Sepal** هي الجزء الخارجي الأخضر الذي يحمي الزهرة قبل تفتحها.
- **Petal** هي الجزء الداخلي الملون الذي يجذب الحشرات للتلقيح.
- هذه القيم الأربعة تُقاس بالسنتيمتر وتُستخدم لتمييز نوع الزهرة.

3. Class (Output): The target or label column represents the flower species:

Class Number	Species Name
0	<i>Iris setosa</i>
1	<i>Iris versicolor</i>
2	<i>Iris virginica</i>

4. Example of the Data: Here's how the first few rows of the dataset look:

features

Sepal length	Sepal width	Petal length	Petal width	Class
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0
7.0	3.2	4.7	1.4	1
6.4	3.2	4.5	1.5	1
6.9	3.1	4.9	1.5	1
6.3	3.3	6.0	2.5	2
5.8	2.7	5.1	1.9	2

label

setosa

versicolo

virginica

```

# 1 ☐ استيراد المكتبات اللازمة
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd

# 2 ☐ تحميل مجموعة بيانات Iris الجاهزة
iris = load_iris()

# 3 ☐ لتسهيل عرضها (DataFrame) تحويل البيانات إلى جدول
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target # إضافة عمود الفئة (النوع)
print("♦ First 10 rows of the Iris dataset:")
print(df.head(10)) # عرض أول 10 صفوف فقط

# 4 ☐ تقسيم البيانات إلى قسمين:
# للتدريب و 30% للاختبار 70%
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=0)

# 5 ☐ بعدد جيران KNN 3 إنشاء نموذج
knn = KNeighborsClassifier(n_neighbors=3)

# 6 ☐ تدريب النموذج باستخدام بيانات التدريب
knn.fit(X_train, y_train)

# 7 ☐ حساب دقة النموذج على بيانات التدريب والاختبار
train_acc = knn.score(X_train, y_train)
test_acc = knn.score(X_test, y_test)
print("\n✔ Model Evaluation:")
print("Training accuracy:", round(train_acc, 3))
print("Testing accuracy:", round(test_acc, 3))

# 8 ☐ تجربة النموذج على عينة جديدة
# تعمل الأرقام [طول الكأس, عرض الكأس, طول البتلة, عرض البتلة]
sample = [[5.1, 3.5, 1.4, 0.2]]
# تنفيذ التنبؤ باستخدام النموذج
prediction = knn.predict(sample)
# عرض نتيجة التنبؤ مع اسم الزهرة
print("\n⊗ Prediction for sample", sample, "is:")
print(iris.target_names[prediction][0])

```

بعد التنفيذ تظهر

```

♦ First 10 rows of the Iris dataset:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1                3.5  ...                0.2         0
1                4.9                3.0  ...                0.2         0
2                4.7                3.2  ...                0.2         0
3                4.6                3.1  ...                0.2         0
4                5.0                3.6  ...                0.2         0
5                5.4                3.9  ...                0.4         0
6                4.6                3.4  ...                0.3         0
7                5.0                3.4  ...                0.2         0
8                4.4                2.9  ...                0.2         0
9                4.9                3.1  ...                0.1         0

```

[10 rows x 5 columns]

```

✔ Model Evaluation:
Training accuracy: 0.962
Testing accuracy: 0.978

```

```

⊗ Prediction for sample [[5.1, 3.5, 1.4, 0.2]] is:
setosa

```

```
# 1 استيراد المكتبات الأساسية
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# 2 تحميل البيانات
cancer = load_breast_cancer()

# 3 تقسيم البيانات إلى تدريب واختبار
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, test_size=0.3, random_state=0)

# 4 عدد جيران = 3 (كمثال) إنشاء نموذج KNN
knn = KNeighborsClassifier(n_neighbors=3)

# 5 تدريب النموذج على بيانات التدريب
knn.fit(X_train, y_train)

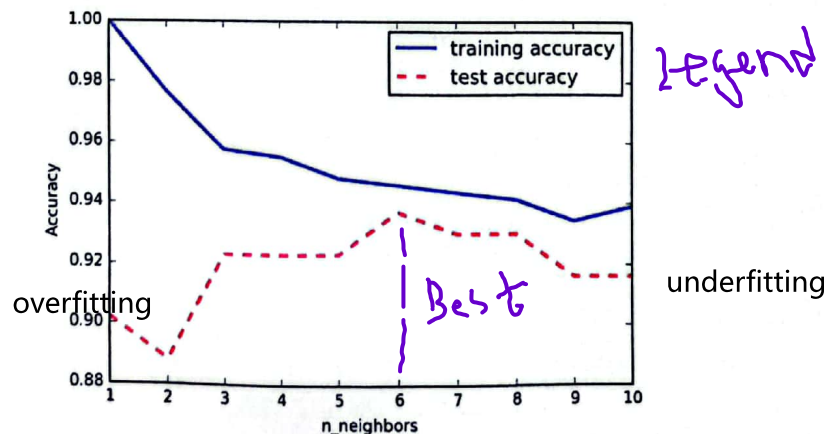
# 6 حساب دقة النموذج على بيانات التدريب والاختبار
train_acc = knn.score(X_train, y_train)
test_acc = knn.score(X_test, y_test)

# 7 عرض النتائج
print("Training accuracy:", train_acc)
print("Testing accuracy:", test_acc)
```

experiment:

In this experiment, we evaluate the accuracy of the K-Nearest Neighbors (KNN) algorithm on both the training and test sets while changing the number of neighbors ($n_neighbors$) from 1 to 10.

The results are shown in a plot where:



- The x-axis represents the number of neighbors.
- The y-axis represents the accuracy for both training and test sets.

When only one neighbor is used ($k = 1$), the model perfectly classifies the training data, showing **overfitting** — it memorizes the training samples but performs worse on unseen data. As the number of neighbors increases, the model becomes **simpler**, and training accuracy decreases. For large values of k (e.g., $k = 10$), the model becomes **too simple (underfitting)** and cannot capture important patterns, so performance also drops.

The **best performance** occurs somewhere in the middle, around $k = 6$, where both training and testing accuracies are well balanced. Even the lowest accuracy (about 88%) is still quite good for

this dataset.

```
# -----
# 📁 Breast Cancer Classification using KNN
# -----

# 1 📁 استيراد المكتبات اللازمة
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

# 2 📁 sklearn تحميل مجموعة بيانات سرطان الثدي الجاهزة من
cancer = load_breast_cancer()

# 3 📁 تقسيم البيانات إلى تدريب واختبار
# للحفاظ على نفس نسبة الفئات في المجموعتين
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target,
    stratify=cancer.target,
    random_state=66
)

# 4 📁 إنشاء قائمتين لتخزين الدقة على بيانات التدريب والاختبار
training_accuracy = []
test_accuracy = []

# تجربة عدد الجيران من 1 إلى 10
neighbors_settings = range(1, 11)

for n_neighbors in neighbors_settings:
    # مع عدد معين من الجيران KNN إنشاء نموذج
    clf = KNeighborsClassifier(n_neighbors=n_neighbors)

    # تدريب النموذج على بيانات التدريب
    clf.fit(X_train, y_train)

    # حساب دقة النموذج على بيانات التدريب
    training_accuracy.append(clf.score(X_train, y_train))

    # حساب دقته على بيانات الاختبار (التعميم)
    test_accuracy.append(clf.score(X_test, y_test))

# 5 📁 رسم العلاقة بين عدد الجيران والدقة
plt.plot(neighbors_settings, training_accuracy, label="Training accuracy")
plt.plot(neighbors_settings, test_accuracy, label="Testing accuracy")
plt.ylabel("Accuracy")
plt.xlabel("Number of Neighbors (k)")
plt.legend()
plt.title("K-Nearest Neighbors on Breast Cancer Dataset")
plt.show()

# 6 📁 تجربة أفضل عدد من الجيران (اختياري)
best_k = 5
clf = KNeighborsClassifier(n_neighbors=best_k)
clf.fit(X_train, y_train)
print(f"📁 Best k = {best_k}")
print("Training accuracy:", clf.score(X_train, y_train))
print("Testing accuracy:", clf.score(X_test, y_test))
```

Plot